Enhancing Code Generation through Retrieval of Cross-Lingual Semantic Graphs

Zhijie Jiang[1], Zejian Shi[2*], Xinyu Gao[2], Yun Xiong[2]
[1]School of Computer, National University of Defense Technology, China
[2]Shanghai Key Laboratory of Data Science, School of Computer Science, Fudan University, China
jiangzhijie@nudt.edu.cn, {zjshi20, yunx}@fudan.edu.cn, xygao23@m.fudan.edu.cn

2024 31st Asia-Pacific Software Engineering Conference (APSEC) | 979-8-3315-3401-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/APSEC65559.2024.00026

***Abstract*—In the field of software engineering automation, code language models have made significant strides in code generation tasks. However, due to the cost of updating knowledge and the issue of hallucinations, code language models (CLMs) face challenges in practical code generation scenarios, making retrieval-augmented code generation a mainstream approach. Existing retrieval-augmented methods only build codebases for a single programming language, which is insufficient to address the lack of monolingual knowledge. To address this, we propose CodeRCSG, a novel cross-lingual retrieval-augmented code generation method. This method constructs a multilingual codebase and creates a unified cross-lingual code semantic graph to capture deep semantic information across different programming languages. By encoding the retrieved code semantic graph with GNN and combining it with input text embeddings, code language models can effectively utilize the transferred cross-lingual programming knowledge to improve the quality of generated code. Experimental results show that CodeRCSG can significantly enhance the code generation capabilities of code language models.**

***Index Terms*—code language models, code generation, retrieval-augmented generation**

I. Introduction

Code generation techniques translate natural language requirements into functional code snippets, significantly speeding up software development and making programming more accessible. Despite substantial advancements by code language models [1]–[6], which have been pre-trained on extensive code corpora, they continue to face challenges such as knowledge updating and avoiding erroneous outputs, commonly referred to as "hallucinations" [7]–[10]. To combat these issues, the retrieval-augmented code generation (RACG) framework [11], [12] has emerged as a pivotal research area in software engineering. RACG begins by constructing a vast codebase of programming practices, using these examples to guide the code generation of language models. This approach not only integrates domain-specific programming knowledge seamlessly, but also substantially reduces the incidence of hallucinations in generated code, enhancing the practical utility of code language models in solving programming problems.

Existing retrieval-augmented code generation methods [13]–[15] typically focus on building a retrieval codebase for a single programming language, which limits their effectiveness in scenarios where monolingual knowledge is insufficient. The success of the RACG framework hinges on its ability to retrieve relevant programming examples from the codebase and integrate this knowledge into the code generation model. This necessitates a codebase that encompasses a broad spectrum of programming needs. However, constructing such comprehensive codebases presents significant challenges, particularly for less commonly used languages in specific domains. For example, while Python is the preferred language for data analysis, creating an equally robust codebase for other languages less dominant in this area remains a daunting task.

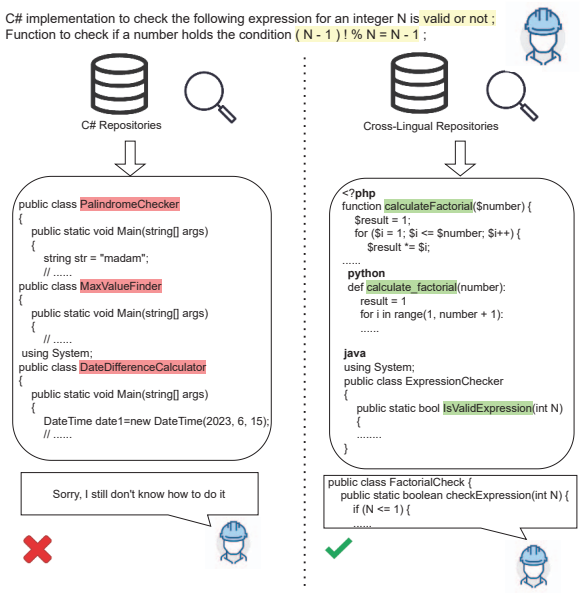

Fig. 1. In reality, if engineers or students encounter difficult problems, searching for solutions in a single language may not be sufficient. Searching for solutions across a broader range of languages often provides more useful assistance.

As illustrated in Figure 1, developers often retrieve useful code snippets from various sources to solve programming problems, mirroring the retrieval-augmented code generation (RACG) process. When suitable examples in their primary language are unavailable, exploring code in other languages can be equally beneficial. This effectiveness stems from the fact that code solving similar problems in different languages often shares comparable functional structures. By understanding and adapting solutions from one language to another,

*Corresponding author

2640-0715/24/$31.00 ©2024 IEEE
DOI 10.1109/APSEC65559.2024.00026

developers not only broaden their technical repertoire but also gain the cross-lingual knowledge for programming. Therefore, leveraging cross-lingual programming knowledge is a feasible approach to improving the capabilities of language models in code generation.

In the above programming scenarios, the transfer of cross-lingual knowledge is essential. The challenge lies in enabling code language models to process and understand this knowledge as adeptly as humans. This process encompasses more than merely bridging syntactic differences and requires grappling with the distinct semantic constructs unique to each programming language. Consider how Python allows direct list slicing, whereas C++ requires the use of standard library functions to create subvectors. This requires models to not only understand the syntax information but also deeply comprehend its underlying functional semantic information.

To overcome the challenges discussed, we proposed CodeRCSG, a novel cross-lingual retrieval-augmented code generation method that significantly enhances language models' code generation capabilities by retrieving cross-lingual code semantic graphs. Initially, we construct a multilingual codebase that includes a variety of programming languages. Each code snippet within this database is paired with a corresponding cross-lingual code semantic graph, derived from the concrete syntax tree. These graphs transcend mere syntactic depictions to encapsulate the deeper semantic information shared across different programming languages. In the code generation phase, when faced with a specific programming problem, the model employs natural language to query the multilingual codebase. It retrieves the most pertinent code snippets and their associated semantic graphs. Subsequently, the embeddings from these graphs, encoded via GNN, are combined with the input text embeddings. This integration supplies specific cross-lingual programming knowledge, enabling code language models not only to produce syntactically accurate code but also to ensure the code fulfills the semantic demands of the task.

To evaluate the effectiveness of CodeRCSG, we conducted extensive experiments using the XLCoST [16] dataset, a benchmark recognized for evaluating code generation models across multiple programming languages. We choose several code language models for our comparative analysis: GPT2-s/m [17], CodeGPT [18], and CodeGEN [19]. The experimental results show that integrating our CodeRCSG results in an average performance improvement of more than 10 points for the EM score and more than 40 points for the BLEU score. These metrics clearly demonstrate CodeRCSG's significant enhancements in code generation capabilities.

In summary, the primary contributions of this paper are as follows:

- We propose CodeRCSG, a novel cross-lingual retrieval-augmented code generation method. This method achieves cross-lingual knowledge transfer by retrieving cross-lingual semantic graphs to cope with scenarios where single-language knowledge is scarce.
- We establish a unified process for constructing and utilizing multilingual code semantic graphs, which enables effective cross-lingual knowledge transfer and improves the quality of generated code by ensuring functional semantic coherence.
- We conduct comprehensive experiments on the seven programming languages of XLCoST to evaluate our CodeRCSG. Experiments results show that CodeRCSG can significantly enhance the code generation capabilities of language models.

II. Background

A. Code Language Model

Code language models [20]–[23] are language models designed to comprehend and generate programming languages. These models are trained on extensive code corpora to learn the structural, syntactical, and semantic aspects of various programming languages. According to different application scenarios, code language models can be broadly categorized into two types: code representation models [24]–[27] and code generation models [17]–[19], [28], [29].

Code Representation Models. Code representation models focus on converting code snippets into dense vector representations that capture both the syntactic and semantic information of programming languages. These models are crucial for applications such as code search [30]–[32] and code clone detection [33], [34], where a deep understanding of the code's underlying meaning and structure is essential. Early code representation models [35] utilize non-contextual word embedding, which are limited in capturing the complexities of programming languages. With the advancements of neural network architectures, deep learning-based code representation models [36]–[38] begin to enhance code comprehension capabilities by understanding contextual nuances. As the field shifts toward pre-training and fine-tuning paradigm, the Transformer-based code representation models [22], [25], [26] have achieved state-of-the-art performance across various code understanding tasks.

Code Generation Models. Code generation models are designed to translate natural language requirements into executable code snippets, effectively automating part of the software development process. These models [17]–[19] also leverage Transformer-based architectures, generate token-level or line-level code snippets by understanding the description or context code provided by the user. Then, notable models like CodeGeeks [2] and Codex [39] have demonstrated the ability to generate function-level code snippets across multiple programming languages from simple prompts. Additionally, emerging models like Code llama [1], DeepSeek Coder [4] and Starcoder2 [40] are pushing the boundaries further by optimizing code quality and adapting to more complex coding tasks, reflecting continuous advancements in this field.

B. Concrete Syntax Tree

Concrete Syntax Tree (CST) [41] is a detailed representation of the syntax of programming languages, where every element

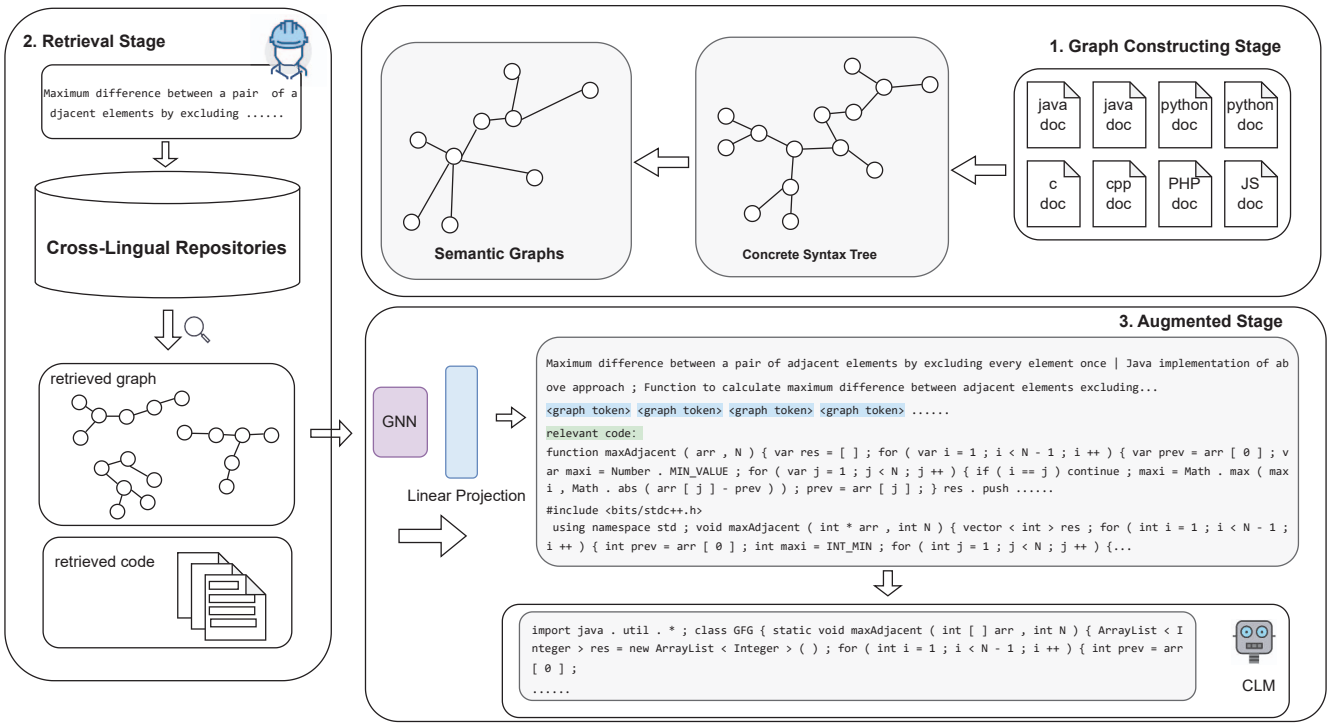

Fig. 2. The figure illustrates the three stages of our methodology. First, we convert the code from its textual form into a more information-rich graph modality. Next, we retrieve semantically similar code from a multilingual corpus. Finally, we provide both the graph and text modalities of the code to the model to aid in generation.

of the source code is explicitly shown, including keywords, operators, and syntactic details that are often abstracted away in other representations like Abstract Syntax Trees (AST) [42]. One of the primary advantages of using CSTs in building cross-lingual semantic graphs is their ability to retain all syntactical elements, which provides a comprehensive foundation for analyzing and comparing the syntax across different programming languages. This feature is crucial when constructing semantic graphs that need to encapsulate not just the operational semantics of the code but also its syntactic nuances, facilitating more accurate and robust cross-lingual knowledge transfer.

III. METHOD

Our overall architecture is comprehensively illustrated in Figure 2, and it consists of three designed phases. First, in the initial phase, we focus on transforming each code snippet into a detailed semantic graph, aiming to provide richer and more precise information for the subsequent generator through the graph modality. Second, in the intermediate phase, we employ advanced semantic similarity algorithms to retrieve the text most semantically similar to the user's query from a multi-modal database, simultaneously extracting the corresponding code and semantic graphs of these texts. Finally, in the third phase, we further utilize Graph Neural Networks (GNNs) to deeply analyze the code semantic graphs, extracting high-level semantic information. These multi-modal data are effectively integrated and fed into code language models, which then generate the target code. This entire flow ensures our system can accurately capture the deep semantics of the code and generate high-quality code that meets the user's requirements.

A. Graph Constructing Stage

First, we parse the code to generate its Concrete Syntax Tree. The CST provides a detailed description of the code's syntactic structure, including each syntactic element and its hierarchical relationships. By analyzing the CST, we can obtain comprehensive syntactic information about the code. One case is shown in Figure 3.

In the process of code analysis and understanding, the size and complexity of the CST often pose a challenge, as not all nodes are crucial for understanding the code's semantics. To more effectively capture the core semantics of the code, we need to extract key nodes from the CST. These nodes typically include variable declarations and usages, function definitions and calls, control flow structures (such as loops and conditional statements), and the definitions and usages of classes and objects. By using predefined rules, we automatically identify and extract these nodes, thereby simplifying the syntax graph and highlighting key semantics. This step not only lays the foundation for the subsequent syntax graph

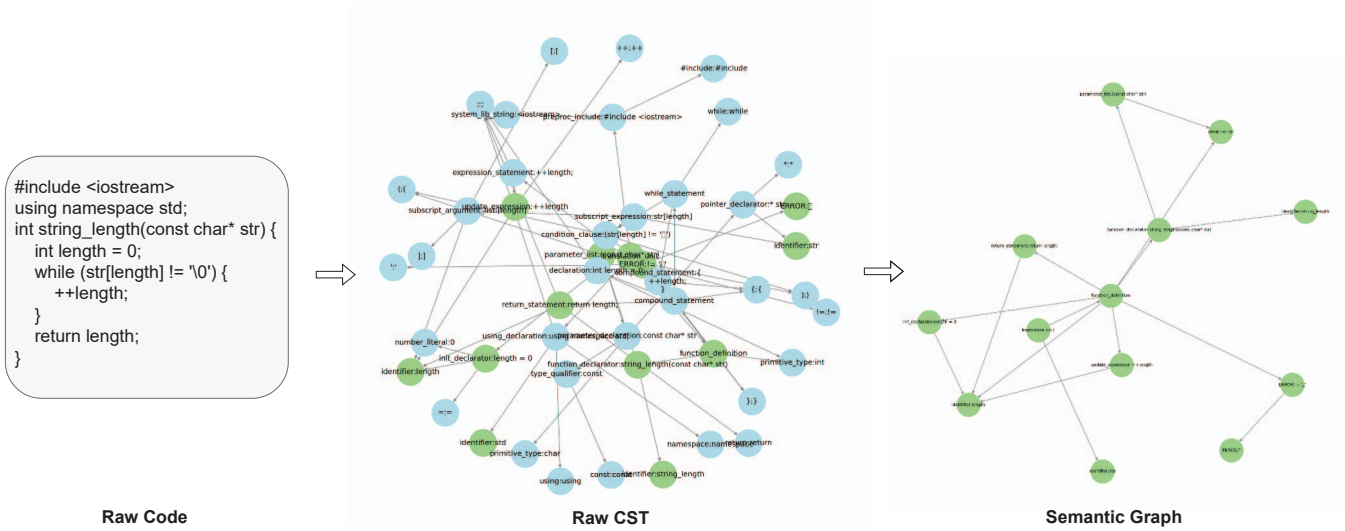

Fig. 3. First, the raw code is converted into a comprehensive CST. Then, significant nodes are selected, and nodes with parent-child relationships are connected to form a syntax graph.

construction but also improves the efficiency of code analysis and understanding.

After extracting the important nodes, constructing a clear and concise syntax graph becomes essential. We connect the extracted nodes according to their relative positions in the CST to form the basic framework of the syntax graph. To simplify the hierarchical structure of the graph, we pay special attention to nodes with ancestor-descendant relationships and directly establish connections between them. Then, by removing redundant nodes and edges, we further simplify and optimize the syntax graph, making it more readable and analyzable. Such a syntax graph not only contains the syntactic information of the code but also reveals its logical structure and semantic relationships. This multi-modal information representation provides a solid foundation for capturing the essential features of the code and supports subsequent tasks such as code analysis, retrieval, and recommendation. By highlighting important nodes and simplifying the graph structure, we enhance both the expressiveness of the syntax graph and its operability and practicality in solving programming problems.

B. Retrieval Stage

Based on the first phase, we have constructed a multilingual, multi-modal retrieval codebase. The core of this retrieval codebase lies not only in containing code snippets in various programming languages but also in equipping each piece of code with a corresponding semantic graph, thus achieving comprehensive semantic understanding and retrieval of code.

When a user inputs a query, the retrieval stage performs the following steps: query encoding, similarity calculating and relevant code retrieval. First, we use pre-trained language models to encode the user's query into a semantic vector. Then, we calculate the cosine similarity between the query vector and the semantic vectors of each code snippet in the retrieval codebase using the following formula:

$$S(q, c_i) = E_{corpus}(c_i) \cdot E_{query}(q) \tag{1}$$

where E_{corpus} and E_{query} are the semantic representation models for corpus and queries respectively.

The retrieval stage not only provides the relevant code snippets but also displays the corresponding semantic graphs, offering users insights into the logical relationships and structure of the code to help them better understand its meaning. This multilingual, multi-modal code retrieval method significantly improves the accuracy and effectiveness of retrieval. By combining code sequence information with semantic graph information, the model can more comprehensively understand the meaning and structure of the code, thus meeting the code generation needs of users in a multilingual environment. By providing the code snippets and semantic graphs from the retrieval results to code language models, we can help them better understand the meaning and structure of the code, thereby generating more accurate and useful code. Therefore, this multilingual, multi-modal code retrieval codebase has important application value in the fields of code retrieval and code generation.

C. Augmented Stage

Following the retrieval and transformation of code snippets into semantic graphs, the next step involves encoding these graphs to extract and utilize deeper, more useful information. We employ GNNs this purpose, as they are adept at learning representations from graph-structured data. GNNs are particularly well-suited for capturing the intricate relationships and dependencies within code, which are represented in the semantic graphs. By processing these graphs, GNNs can learn high-level features that encapsulate both the syntactic structure and the semantic meaning of the code. This process involves

iteratively updating the node and edge representations in the graph to reflect their context and connections, resulting in a rich, embedded representation of the code. Following the GNN processing, a pooling operation is applied to the output to condense it into a fixed number of tokens, referred to as code tokens.

Once the semantic graphs are encoded by the GNN, the next step is to align the information from the graph modality with the natural language modality. This alignment is achieved through a projection mechanism, which maps the graph embeddings into the same feature space as the language embeddings produced by the code language models. The projection ensures that the two different modalities, graph-based code information and text-based language information, which can be effectively integrated and utilized by the code generator. The process is as follows: First, the semantic graphs are processed by GNNs, generating rich embeddings that capture the deeper features of the code. Then, the encoded graph representations are projected into the same feature space as the language model embeddings, ensuring compatibility. Finally, the aligned embeddings are integrated into the code generator, enabling them to leverage both structural insights from the code graphs and contextual knowledge from natural language.

Then we integrate multiple sources of information to generate high-quality code. This process starts with the initial user query and includes the retrieved code snippets in text form as well as the high-level representations of the code graphs. These diverse inputs are combined to form a comprehensive context, which is fed into a code language model. To achieve this, we use autoregressive models, which are widely applied in state-of-the-art language generation tasks. These models generate code token by token, using previously generated tokens as additional context. The combined inputs, including the original query, retrieved code text, and graph-based embeddings, provide the model with a rich and multifaceted understanding of the code generation task.

To train this integrated framework, we employ a cross-entropy loss function, which is used to jointly fine-tune the parameters of the CLM, GNN, and projection mechanism. The training process includes several steps: First, combining the original query, retrieved code snippets, and high-level graph representations into a unified input sequence. Second, feeding this combined input into the autoregressive model, which generates code token by token, ensuring that each token prediction considers both textual and structural information. Finally, using the cross-entropy loss function to measure the difference between the predicted tokens and the actual tokens, providing gradients for updating the model parameters. By minimizing the cross-entropy loss, the model learns to generate accurate and contextually relevant code. The training process jointly optimizes the CLM, GNN, and projection layer, ensuring that all components are fine-tuned to work together effectively. The cross-entropy loss function is as follows:

TABLE I
THIS TABLE PRESENTS THE STATISTICS OF THE DATASET.

	C++	Java	Py	C#	JS	PHP	C
split							
train	9797	9623	9263	9345	8590	3087	463
test	492	494	472	491	475	158	60
valid	909	911	887	899	886	308	54
total	11198	11028	10622	10735	9951	3553	574
stats							
# lines	32.45	34.93	20.54	35.64	26.47	23.23	31.5
# tokens	205	227	188.5	215.3	184.6	163.5	198
# SN/PR	9.52	9.42	8.51	9.33	8.2	5.81	7.77

$$Loss = -\frac{1}{T}\sum_{t=1}^{T}\sum_{i=1}^{V} y_t(i) \cdot \log(p_t(i)) \tag{2}$$

where T is the length of the sequence, V is the size of the vocabulary, and $y_t(i)$ and $p_t(i)$ are the probabilities of the i-th word in the vocabulary at time step t according to the true distribution and the model's predicted distribution, respectively.

Through this comprehensive training approach, our method effectively integrates the original query, retrieved code, and graph-based representations. The result is a powerful and capable CLM that excels in code generation tasks, benefiting from the rich contextual information provided by both text and graph modalities.

IV. EXPERIMENT SETUP

A. Datasets

We used XLCoST [16], a machine learning benchmark dataset that includes fine-grained parallel data for seven commonly used programming languages (C++, Java, Python, C#, Javascript, PHP, C) and a natural language (English). The data is parallel across the seven languages, including both code snippet and program levels. This means that for a given program in one language, the dataset contains the same program in up to six other programming languages. Each program is divided into several code snippets, and the programs in all languages are aligned at the snippet level. For the multi-language setup, we use the union of the training, validation, and test sets from multiple languages as the retrieval pool. In the single-language experiment setup, we use the training, validation, and test sets of the corresponding programming language as the retrieval pool, and retrieve relevant code based on the semantic similarity of natural language.

B. Base Models

Retrieve base model: In the retriever section, we have set up UAE-Large-V1 [43], which introduces angle optimization in complex space, effectively mitigating the adverse effects of saturation areas in the cosine function that can impede

gradients and hinder the optimization process. We treat the natural language in the dataset as the key for retrieval, use embeddings to search for the most semantically similar natural language, and then extract the corresponding code and code diagrams.

Generator base model:

We have configured various autoregressive language models with different parameter sizes and pre-training methods. GPT-2 [17], proposed by OpenAI, is a self-supervised Transformer model pre-trained on large English datasets. CodeGPT [18], by CodeXGLUE, is similar to GPT-2 but trained on code. CodeGen [19], by Salesforce, describes code generation as a multi-turn conversation, enhancing performance. In our experiments, we use GPT-2's small and medium versions, CodeGPT-small-java-adapted, and CodeGen's codegen-350M-mono versio

C. Evaluation Metrics

Consistent with previous work, we use the following metrics to evaluate the quality of code generation: Exact Match (EM) and BLEU [44]. EM is a metric that measures whether the generated code exactly matches the reference code. BLEU is an n-gram-based metric that measures the similarity between the generated code and the reference code, with scores ranging from 0 to 1, where a score closer to 1 indicates higher similarity between the generated code and the reference code.

D. Implementation Details

As described in SectionIII, CodeRCSG consists of three independent components. For dense retrieval, we use the FAISS database to store embeddings and utilize the LangChain framework for similarity matching. We construct code semantic graphs using the tree-sitter Python version and initialize node representations with MiniLM [45]. We set the generator's block size to 1024. For training, we set the batch size to 8, the number of epochs to 10, and the learning rate to 1e-5. All experiments were conducted on four NVIDIA GeForce RTX 3090 GPUs.

V. EXPERIMENT RESULT

- RQ1: Main Experiment and Impact on Performance
- RQ2: Impact of Retrieval Database
- RQ3: impact of code tokens
- RQ4: Ablation Study: Impact of Graph Structure and Text

The research focuses on optimizing retrieval-augmented code generation methods by exploring experimental setups, database quality, parameter adjustments, and the roles of code semantic graph structure and text information. Through comparative analysis and ablation experiments, the aim is to enhance the performance and effectiveness of code generation models in solving programming problems.

A. RQ1: overview of performance

The experimental results demonstrate that the introduction of the GCN architecture in CodeRCSG significantly improves code generation performance across various models (GPT2-s, GPT2-m, CodeGPT, CodeGEN). In all experimental languages (C, C++, C#, Java, JavaScript, PHP, Python), CodeRCSG effectively leverages the structural information and multilingual features of code by incorporating cross-lingual code semantic graphs, thereby enhancing the accuracy and quality of code generation models. For instance, in languages like C and C++ with their lower-level structures and simpler syntax, the GCN effectively captures semantic information, thereby boosting the performance of the generation models. In higher-level languages such as C# and Java, all models show significant performance improvements after the introduction of CodeRCSG, with CodeGPT and GPT2-m exhibiting greater adaptability and accuracy. Experimental results for JavaScript and PHP also indicate performance enhancements with the integration of CodeRCSG, particularly in BLEU scores. Despite the flexibility and dynamic typing of these languages posing challenges for code generation, the GCN significantly improves the quality of generated code by capturing key semantic information in the code. Additionally, the performance of all models in Python also improves with the integration of CodeRCSG, demonstrating the GCN's effectiveness in capturing dynamic types and semantic relationships.

Furthermore, the experiments reveal that incorporating multilingual features into the retrieval codebase greatly enhances the performance of code generation models. By introducing cross-lingual code semantic graphs, the models can share and utilize semantic information across different programming languages, which not only enhances the applicability and accuracy of code generation but also significantly improves the quality of generated code. These results indicate the crucial role of CodeRCSG in improving code generation quality and accuracy. By incorporating cross-lingual code semantic graphs, the models can better understand and generate code that meets user requirements. This approach is effective for both statically typed and dynamically typed languages, ensuring that the generated code aligns more closely with the user's original description and intended functionality. Future research could further explore integrating domain-specific knowledge into the GCN, such as syntax-aware embeddings or domain-specific attention mechanisms, to further enhance the performance and applicability of code generation. By optimizing and integrating appropriate GNN models, the accuracy and adaptability of code generation can be significantly improved, better meeting practical application needs.

B. RQ2: impact of retrieval database

This experiment systematically compared the code generation capabilities of models like GPT-2 and CodeGPT when supported by single-language and multilingual codebases. The results show that multilingual codebases offer significant advantages. The rich corpus of multilingual codebases helps models grasp diverse coding styles and expressions, enhancing flexibility and accuracy in code generation. Additionally, cross-language knowledge transfer allows models to quickly adapt to new programming tasks, reducing errors and re-

TABLE II

THE TABLE ILLUSTRATES THE IMPACT OF INTRODUCING CODERCSG ON THE PERFORMANCE OF CODE GENERATORS, ENCOMPASSING A COMPREHENSIVE EVALUATION ACROSS SEVEN PROGRAMMING LANGUAGES AND FOUR DIFFERENT MODELS. THE EXPERIMENTAL RESULTS DEMONSTRATE THAT THE INTEGRATION OF CODERCSG SIGNIFICANTLY AND SUBSTANTIALLY ENHANCES CODE GENERATION PERFORMANCE, VALIDATING ITS EFFECTIVENESS IN IMPROVING THE QUALITY OF CODE GENERATION. THE TYPE OF GNN IS GCN.

Lang.	GPT2-s		+CodeRCSG		GPT2-m		+CodeRCSG		CodeGPT		+CodeRCSG		CodeGEN		+CodeRCSG	
	EM	BLEU	EM	BLEU	EM	BLEU	EM	BLEU	EM	BLEU	EM	BLEU	EM	BLEU	EM	BLEU
C	0.00	30.67	11.76	83.64	0.00	36.41	14.65	88.67	0.00	30.27	**17.65**	84.69	0.00	37.30	13.73	**89.29**
C++	0.00	41.50	14.12	82.01	0.11	42.65	16.94	84.20	0.00	41.33	**17.05**	83.74	0.00	41.95	15.73	**84.82**
C#	0.00	39.61	16.35	84.63	0.00	39.56	**19.13**	**86.22**	0.00	39.96	18.68	85.03	0.11	39.58	16.24	85.83
Java	0.00	38.98	7.57	81.39	0.00	40.23	9.00	**83.60**	0.00	40.72	**18.78**	81.77	0.00	41.45	7.68	83.42
Javascript	0.00	31.69	16.70	79.69	0.00	32.97	14.00	**81.70**	0.00	29.11	**17.38**	80.24	0.00	35.06	15.91	80.66
PHP	0.00	33.97	15.26	85.60	0.32	39.92	**18.18**	**88.42**	0.00	39.74	16.56	86.01	0.00	31.17	15.91	86.50
Python	0.00	36.33	5.98	74.60	0.00	35.44	**7.33**	**76.39**	0.00	37.52	6.76	74.76	0.00	35.96	6.99	75.06

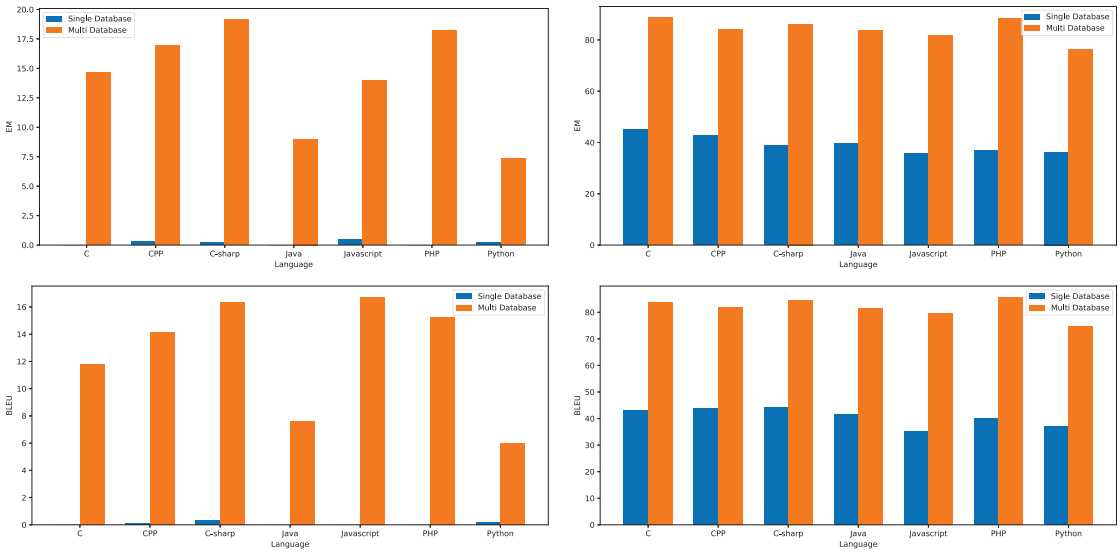

Fig. 4. Shows the impact of different models in different retrieval codebases. Across multiple metrics, enhancing the model's capabilities using multiple language codebases is much more effective than using a single language codebase.

dundancy in the code, which greatly improves overall code generation performance.

In contrast, single-language codebases are limited in corpus size, restricting the diversity and innovation in the models' code generation, thus affecting code quality and applicability. Experimental data further confirms that GPT-2 and CodeGPT, when enhanced by multilingual codebases, excel in code accuracy, fluency, and diversity, with significant improvements in BLEU and EM metrics. This demonstrates that multilingual codebases not only strengthen the models' ability to handle complex coding tasks but also improve the quality and consistency of generated code.

Therefore, future research and applications should further explore the potential of multilingual codebases to advance the development of language models in the field of code generation.

C. RQ3: impact of code tokens

As shown in the Figure 5, with the increase in the number of code tokens, the EM (Exact Match) metric exhibits a trend of initially rising and then falling. This trend can be divided into three stages: As the number of tokens increases, the rich contextual information promotes the generation of more accurate code segments, causing the EM value to rise. After reaching a certain threshold, the model efficiently utilizes the ample information to generate highly matched code, leading the EM value to peak. However, in the later stage, an excessive number of tokens introduce noise and redundant information, causing information overload for the model, which in turn

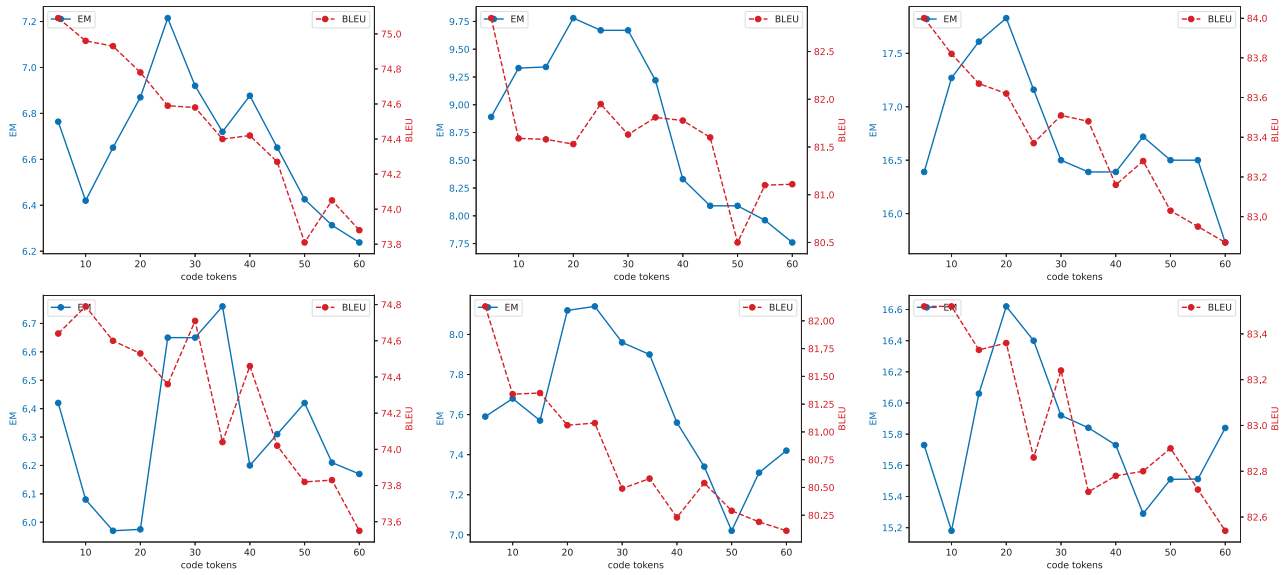

Fig. 5. The above trends show the performance of CodeGPT in code generation for Python, Java, and C++ as the value of n changes. The trends below show the performance variation of GPT-2 in code generation.

decreases the accuracy of the generated code, and the EM value starts to decline.

Unlike the EM metric, the BLEU score continuously declines as the number of code tokens increases. This reflects that BLEU not only considers exact matches but also values the diversity and fluency of the generated code. The injection of knowledge in the form of soft prompts may mislead the generative model, leading to a deterioration in BLEU scores. As the number of tokens increases, the complexity of the code rises, and although it may become more diverse, its strict consistency with the reference code diminishes, resulting in a lower BLEU score. Additionally, the error accumulation effect and the rigorous requirements specific to code exacerbate this trend.

D. RQ4: impact of graph structure and text

This study examines the individual roles of code semantic graph structure and text information in retrieval-augmented code generation through ablation experiments. Both components are crucial: graph structure captures hierarchical relationships and semantic dependencies, while text information provides contextual cues and syntactic details. Removing either significantly impairs code generation accuracy and relevance. Hybrid models integrating both representations often outperform single-modality approaches, enhancing code generation accuracy and adaptability. Understanding their synergies optimizes model architectures for improved performance in solving programming problems.

VI. Threats to Validity

Dataset Quality and Diversity. The quality and diversity of the code repositories used to build our cross-lingual code semantic graph (CSG) play a crucial role in the effectiveness of our approach. If the dataset is biased towards specific programming languages, paradigms, or coding styles, the generated code may not generalize well to other contexts or languages. Ensuring a comprehensive and representative dataset is essential for mitigating this threat.

Evaluation Metrics. The metrics used to evaluate the performance of CodeRCSG might not fully capture the nuances of code generation quality. Standard metrics such as BLEU or ROUGE scores, while useful, may not adequately reflect the correctness, readability, and efficiency of the generated code. Incorporating human evaluation and task-specific metrics could provide a more holistic assessment of the model's performance.

Generalizability to Real-World Scenarios. The experiments conducted in this study may not encompass all possible real-world programming scenarios. The effectiveness of CodeRCSG in practical software development environments, with varying project requirements and coding standards, remains to be thoroughly evaluated. Future work should involve extensive testing in diverse real-world settings to validate the robustness and utility of the framework.

VII. Related Works

A. Retrieval-Augmented Code Generation

Retrieval-augmented methods enhance LLMs by retrieving relevant external knowledge, thereby improving their performance in knowledge-intensive domains. Classic RA-LM methods, such as DocPrompt [11] and Recoder [12], integrate codebases or document repositories to provide rich contextual information for code models, significantly enhancing the quality of generated target code. To address this challenge, APICoder [46] proposed an innovative framework that enables

TABLE III
THE TABLE PRESENTS THE RESULTS OF THE ABLATION STUDY. IT CAN BE SEEN THAT TEXTUAL INFORMATION STILL HAS A SIGNIFICANT IMPACT ON THE RESULTS. HOWEVER, INTRODUCING THE STRUCTURAL INFORMATION FROM THE GRAPH LEADS TO EVEN BETTER PERFORMANCE.

Lang.	C		C++		C#		Java		Javascript		PHP		Python	
	EM	BLEU	EM	BLEU	EM	BLEU	EM	BLEU	EM	BLEU	EM	BLEU	EM	BLEU
GPT2-m	14.65	88.67	16.94	84.19	19.13	86.22	9.01	83.59	13.99	81.70	18.19	88.41	7.32	76.39
w/o graph	15.88	77.51	16.94	84.80	17.02	86.22	9.00	83.52	12.19	80.57	15.26	86.40	5.75	74.69
w/o text	0.00	30.20	0.00	43.21	0.00	40.72	0.00	40.70	0.00	35.20	0.00	35.00	0.00	37.73
CodeGPT-java	17.64	84.69	17.05	83.73	18.68	85.02	8.78	81.76	17.38	80.23	16.55	86.01	6.76	74.76
w/o graph	15.69	74.81	15.07	84.20	19.91	86.67	9.88	83.82	13.54	80.33	14.29	86.31	6.43	75.25
w/o text	0.00	29.66	0.00	39.56	0.00	34.68	0.00	37.48	0.00	27.60	0.00	29.36	0.00	32.49

LLMs to adapt to private codebase. This framework first uses the APIRetriever component to accurately retrieve task-relevant APIs, followed by the APICoder module which generates corresponding code snippets based on the documentation of these APIs.

Furthermore, Repocoder [47] developed an iterative generation and retrieval mechanism that achieves more refined code completion at the repository level. Additionally, the ARKS [13] project enhances code generators' performance by building a more dense knowledge base and incorporating active retrieval strategies. These research advancements not only demonstrate the potential of RAG technology in the field of code generation but also provide valuable insights and guidance for future studies.

B. Code Representation

Graph representation has been employed in various software engineering tasks [48], [49]. These methods typically convert source code into abstract syntax trees (ASTs), control flow graphs (CFGs), or data flow graphs (DFGs), then use GNNs to learn the code's embedded representations. GNNs have become powerful tools for learning graph-structured code representations. We have developed GNNs specifically for code property graphs (CPGs), iteratively propagating information to learn high-level code representations [49], [50]. To improve the quality of code representation, some studies have adopted multi-view contrastive learning methods, enhancing the model's understanding of code semantics by comparing representations from different views [50]. We have proposed graph matching network-based methods for code semantic learning, which help improve the efficiency of code clone detection [51]. Graph models are also used in other aspects of software engineering, such as modeling design patterns, deployment topologies, and development processes [52]. Graph transformations serve as fundamental tools in model-based software development, including domain-specific language engineering. Despite the progress in graph-based code representation, how to effectively extract and utilize graph structure information from large code bases remains a challenge

VIII. CONCLUSION

This paper addresses the challenges faced by CLMs in code generation tasks, such as high knowledge update costs, hallucination issues, difficulties in constructing retrieval codebases, and semantic gaps, by proposing an innovative method CodeRCSG. CodeRCSG successfully captures deep semantic information between codes in different programming languages through the construction of cross-lingual code semantic graphs and effectively integrates the retrieved code semantic knowledge into the code generation model using GNNs encoding technology, thereby significantly improving the accuracy and generalization capability of code generation.

ACKNOWLEDGMENT

This work is supported by Shanghai Science, Technology Development Fund No.22dz1200704.

REFERENCES

[1] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[2] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, Z. Wang, L. Shen, A. Wang, Y. Li *et al.*, "Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x," *arXiv preprint arXiv:2303.17568*, 2023.

[3] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.

[4] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: When the large language model meets programming–the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.

[5] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, "Codet5+: Open code large language models for code understanding and generation," *arXiv preprint arXiv:2305.07922*, 2023.

[6] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: Source code is all you need," *arXiv preprint arXiv:2312.02120*, 2023.

[7] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *arXiv preprint arXiv:2311.05232*, 2023.

[8] Z. Xu, S. Jain, and M. Kankanhalli, "Hallucination is inevitable: An innate limitation of large language models," *arXiv preprint arXiv:2401.11817*, 2024.

[9] F. Liu, Y. Liu, L. Shi, H. Huang, R. Wang, Z. Yang, and L. Zhang, "Exploring and evaluating hallucinations in llm-powered code generation," *arXiv preprint arXiv:2404.00971*, 2024.

[10] Y. Tian, W. Yan, Q. Yang, Q. Chen, W. Wang, Z. Luo, and L. Ma, "Codehalu: Code hallucinations in llms driven by execution-based verification," *arXiv preprint arXiv:2405.00253*, 2024.

[11] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, "Docprompting: Generating code by retrieving the docs," *arXiv preprint arXiv:2207.05987*, 2022.

[12] M. R. Parvez, W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Retrieval augmented code generation and summarization," *arXiv preprint arXiv:2108.11601*, 2021.

[13] H. Su, S. Jiang, Y. Lai, H. Wu, B. Shi, C. Liu, Q. Liu, and T. Yu, "Arks: Active retrieval in knowledge soup for code generation," *arXiv preprint arXiv:2402.12317*, 2024.

[14] J. Chen, X. Hu, Z. Li, C. Gao, X. Xia, and D. Lo, "Code search is all you need? improving code suggestions with code search," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[15] M. Liu, T. Yang, Y. Lou, X. Du, Y. Wang, and X. Peng, "Codegen4libs: A two-stage approach for library-oriented code generation," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 434–445.

[16] M. Zhu, A. Jain, K. Suresh, R. Ravindran, S. Tipirneni, and C. K. Reddy, "Xlcost: A benchmark dataset for cross-lingual code intelligence," *arXiv preprint arXiv:2206.08474*, 2022.

[17] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[18] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang *et al.*, "Codexglue: A machine learning benchmark dataset for code understanding and generation," *arXiv preprint arXiv:2102.04664*, 2021.

[19] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.

[20] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, 2021.

[21] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," *arXiv preprint arXiv:2103.06333*, 2021.

[22] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unixcoder: Unified cross-modal pre-training for code representation," *arXiv preprint arXiv:2203.03850*, 2022.

[23] C. Niu, C. Li, V. Ng, J. Ge, L. Huang, and B. Luo, "Spt-code: Sequence-to-sequence pre-training for learning source code representations," in *Proceedings of the 44th international conference on software engineering*, 2022, pp. 2006–2018.

[24] A. Kanade, P. Maniatis, G. Balakrishnan, and K. Shi, "Learning and evaluating contextual embedding of source code," in *International conference on machine learning*. PMLR, 2020, pp. 5110–5121.

[25] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[26] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.

[27] X. Wang, Y. Wang, F. Mi, P. Zhou, Y. Wan, X. Liu, L. Li, H. Wu, J. Liu, and X. Jiang, "Syncobert: Syntax-guided multi-modal contrastive pre-training for code representation," *arXiv preprint arXiv:2108.04556*, 2021.

[28] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 2022, pp. 1–10.

[29] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-t. Yih, L. Zettlemoyer, and M. Lewis, "Incoder: A generative model for code infilling and synthesis," *arXiv preprint arXiv:2204.05999*, 2022.

[30] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," *arXiv preprint arXiv:1909.09436*, 2019.

[31] S. Sachdev, H. Li, S. Luan, S. Kim, K. Sen, and S. Chandra, "Retrieval on source code: a neural code search," in *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2018, pp. 31–41.

[32] J. Cambronero, H. Li, S. Kim, K. Sen, and S. Chandra, "When deep learning met code search," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 964–974.

[33] M. White, M. Tufano, C. Vendome, and D. Poshyvanyk, "Deep learning code fragments for code clone detection," in *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*, 2016, pp. 87–98.

[34] C. Fang, Z. Liu, Y. Shi, J. Huang, and Q. Shi, "Functional code clone detection with syntax and semantics fusion learning," in *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*, 2020, pp. 516–527.

[35] U. Alon, M. Zilberstein, O. Levy, and E. Yahav, "Code2vec: Learning distributed representations of code," *Proc. ACM Program. Lang.*, vol. 3, no. POPL, pp. 40:1–40:29, Jan. 2019. [Online]. Available: http://doi.acm.org/10.1145/3290353

[36] X. Gu, H. Zhang, and S. Kim, "Deep code search," in *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 933–944.

[37] Y. Wan, J. Shu, Y. Sui, G. Xu, Z. Zhao, J. Wu, and P. Yu, "Multi-modal attention network learning for semantic source code retrieval," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 13–25.

[38] J. Gu, Z. Chen, and M. Monperrus, "Multimodal representation for neural code search," in *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2021, pp. 483–494.

[39] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[40] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei *et al.*, "Starcoder 2 and the stack v2: The next generation," *arXiv preprint arXiv:2402.19173*, 2024.

[41] G. Rakić and Z. Budimac, "Introducing enriched concrete syntax trees," *arXiv preprint arXiv:1310.0802*, 2013.

[42] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 783–794.

[43] X. Li and J. Li, "Angle-optimized text embeddings," *arXiv preprint arXiv:2309.12871*, 2023.

[44] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.

[45] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," *Advances in Neural Information Processing Systems*, vol. 33, pp. 5776–5788, 2020.

[46] D. Zan, B. Chen, Z. Lin, B. Guan, Y. Wang, and J.-G. Lou, "When language model meets private library," *arXiv preprint arXiv:2210.17236*, 2022.

[47] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, "Repocoder: Repository-level code completion through iterative retrieval and generation," *arXiv preprint arXiv:2303.12570*, 2023.

[48] M. Saad and T. Sharma, "Concord: Towards a dsl for configurable graph code representation," *arXiv preprint arXiv:2401.17967*, 2024.

[49] J. Liu, J. Zeng, X. Wang, and Z. Liang, "Learning graph-based code representations for source-level functional similarity detection," in *ICSE*, 2023.

[50] R. Wu, Y. Zhang, and L. Chen, "Improving code representation learning via multi-view contrastive graph pooling for abstract syntax tree," in *International Conference on Collaborative Computing: Networking, Applications and Worksharing*. Springer, 2023, pp. 242–261.

[51] D. Yu, Q. Yang, X. Chen, J. Chen, and Y. Xu, "Graph-based code semantics learning for efficient semantic code clone detection," *Information and Software Technology*, vol. 156, p. 107130, 2023.

[52] B. Casey, J. Santos, and G. Perry, "A survey of source code representations for machine learning-based cybersecurity tasks," *arXiv preprint arXiv:2403.10646*, 2024.